Practice Guide 03_02: Autoencoders, Time and Space

Aprendizagem Aplicada à Segurança

Mário Antunes

October 23, 2026

Objective

Build an autoencoder detector in **Keras 3 on the JAX backend** and compare it with its linear twin (PCA). See why the size of the code matters and why it cannot be tuned without labelled anomalies. Then build a windowed detector for a time series, compare it with a one-liner, and see how point-adjusted metrics inflate results.

- **Lecture:** `slides/slides_03_02.pdf`; **demos:** `notebooks/02-anomaly/` 04 to 06.
- **Notebook to complete:** `guide_03_02.ipynb` (this folder). Cells marked `TODO` are yours; `check` cells tell you whether you are right. Helpers: `notebooks/02-anomaly/anomlib.py` and `deeplib.py`.
- **Data:** KDD Cup 99 (as in guide 03_01), NAB series `datasets/art_daily_*.csv`, Example 2 (a simulated water plant, generated).
- **Time:** about 2 hours (suggested: A 25 min, B 25, C 30, D 15, questions 25). Training runs on a CPU (the whole notebook takes about 1 minute).
- **Environment:** `make venv`; Keras uses JAX (`KERAS_BACKEND=jax` is set in the first cell). No TensorFlow, no PyTorch.

Background in one page

Autoencoder. An encoder f maps $\mathbf{x}$ (d values) to a code $\mathbf{h}$ ($k \ll d$), a decoder g rebuilds $\hat{\mathbf{x}} = g(f(\mathbf{x}))$. Trained on normal rows to minimise $\frac{1}{n}\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$; anomaly score $s(\mathbf{x}) = \frac{1}{d}\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. A **linear** AE learns the PCA subspace; ReLU layers let it bend.

Denoising AE. Input $\mathbf{x} + \epsilon$, target $\mathbf{x}$: the model learns the shape of normal data rather than copying points.

Hyper-parameters without labels. Code size, epochs, noise: a one-class setting has

no labelled anomaly to choose them; the validation loss of *normal* data is the only honest signal.

Windows. A contextual anomaly is visible only in context: score windows $\mathbf{x}_{i..i+w-1}$. With a window AE, the squared error of window i at step t is $e_{i,t}$; the score of step t is the mean of $e_{i,t}$ over the windows that contain t.

Seasonal difference. $|x_t - x_{t-P}|$ with P the period (one day = 288 samples of 5 minutes).

Point adjust. If one point of an anomalous segment is flagged, the whole segment counts as detected. It inflates F1, even for random scores.

Tasks

Part	What you do	Check
A	Implement `build_ae` and `recon_error`; train (given); compare with PCA	shapes, table
B	Implement `sweep(codes)`: AUPRC and validation MSE per code size; denoising (given)	table
C	Implement `seasonal_diff` and `per_step_score`; conv AE on NAB (given)	small arrays, table
D	Implement `point_adjust`; compare F1 for a random score	small arrays

Tips. `model.predict(X, verbose=0)` returns the reconstruction. Seeds are fixed with `keras.utils.set_random_seed`; small differences between runs on different machines are normal (report "typical" numbers).

Questions

1. **(A1)** Is the AE better than PCA with the same number of components? What do the ReLU layers add?
2. **(B1)** Which code size has the best AUPRC, and which the lowest validation MSE? Could you have chosen the best size without the test attacks?
3. **(B2)** Does the denoising AE help on KDD99? Why might it help more on raw signals?
4. **(C1)** Do the seasonal difference and the conv AE find the two anomalies? Where do the false alarms of the seasonal difference come from?

5. **(C2)** Why must the window cover a whole day? What happens with a window of a few hours?
6. **(C3)** What would you try first on a new time series: a one-liner or a deep model? Why?
7. **(D1)** Why does a random score reach a high point-adjusted F1? Which metrics would you report instead?

Model	AUPRC	Recall at 1% FPR
AE, code 8		
PCA, 8 components		
AE, best code (which?)		
denoising AE		

Optional challenges

- **Variational AE and Deep SVDD:** use `dl.VAE()` and `dl.DeepSVDD()` from `deeplib.py` on KDD99 with three seeds. Report mean and standard deviation of the AUPRC.
- **LSTM:** replace the conv AE in part C by `dl.lstm_ae(w, 1)` with $w = 96$ and then $w = 288$ (slower). Does the window length matter more than the architecture?
- **Space:** on Example 2, fit PCA with 2 components on the normal run and find which of the three events it detects (notebook 06). Why does a per-sensor z-score miss the frozen sensor?

Reference solution

The solved notebook, with the answers to the questions and the numbers of a full run, is `solution/guide_03_02.ipynb`.